# Digital twins for software engineers: a survey through a potted plant

## Scope

This survey maps the digital twin (DT) literature for one reader: a software engineer at home with message brokers, containers and continuous integration (CI), who has been handed a digital twin to build or assess. It covers what separates a twin from the systems you already ship, the architectural and deployment choices on offer, the platforms that claim to spare you the work, the failure modes that recur, and the exemplars worth cloning. It excludes the physics inside the models, manufacturing process detail, control theory, and adoption economics. One example anchors every section: a potted plant, a soil-moisture probe, and a small pump.

## 1. The connection is the twin, not the model

The literature's sharpest distinction is about coupling, not fidelity. Kritzinger et al. grade digital counterparts by level of integration, separating a Digital Model from a Digital Shadow and from a Digital Twin, and find that work at the highest stage is the scarcest [1]. Ellwein et al. restate the criterion as the degree of *automated* integration: a Digital Model needs a person to carry changes across by hand [2]. A Digital Shadow has "an automated one-way data flow" from the physical object to the digital one [3], and shadow and twin are treated as a paired distinction in manufacturing [4].

Apply that to the pot. A service that ingests moisture readings and draws a chart is a digital shadow. It becomes a twin when the same service decides to run the pump, and the soil gets wetter because the software said so.

Do not expect the term to carry that meaning on someone else's requirements document. Definitions vary widely enough that the variance is itself a reported problem [5], [6], and a twin reflects only the properties that matter "within a specific application context" [7] — which is what licenses you to model a living plant as three numbers. The sources also disagree on the central point: the Digital Twin Consortium definition quoted by Bhandal et al. asks for a virtual representation "synchronized at a specified frequency and fidelity" and names no automated flow back to the physical side [8]. Under Kritzinger, your read-only moisture logger is not a twin; under the Consortium, it is.

## 2. Architecture: familiar shapes, one unfamiliar edge

The software architecture community has claimed twins as its own subject, with a growing body of peer-reviewed architectural solutions [9]. Two pattern catalogs offer shapes to instantiate rather than principles to derive: one for twin architecture [10], one for giving twin models behaviour rather than structure alone [11].

The live architectural split is whether a twin is data or a process. Commercial platforms represent twins as passive JSON entities, while research increasingly treats them as orchestrated microservices [12] — realised through microservices and serverless functions across the cloud-to-edge continuum [13], or on Kubernetes with ordinary cloud-native tooling [14]. Deployments span centralised cloud platforms, edge-deployable brokers and containerised twins, and managing them across that range remains under-explored [15].

The unfamiliar edge is that one of your services now writes to a pump. That makes the twin a security boundary: an adversary can attack from the digital side to reach physical assets, and the attack surface is large because the paradigm joins two worlds [16]. Near-real-time twin traffic can carry critical signals, and adding a twin expands an already growing attack surface [17].

## 3. Buy, compose, or write it

Reuse is the field's answer to twin construction cost. Talasila et al. argue that building a twin is hard because of the variety of assets, models, data and services to marshal, and propose a generic platform assembling twins from reusable components offered as a service [18], later demonstrating composition of twins from parts [19]. Gil et al. survey the open-source frameworks through a case study [20], and this Digital-Twin-as-a-Service (DTaaS) framing recurs independently elsewhere [21], [22].

That last survey supplies vocabulary worth adopting: *twin fidelity* is how completely the twin represents its counterpart's state and structure, *twin granularity* is which parts get modelled at all [22]. "Should the plant model include root depth" is a granularity question; "how close should the predicted moisture curve sit to the probe" is a fidelity one. Both are easier to answer than "how good should the model be".

## 4. Where these projects actually fail

Three failure modes recur, and each is a familiar distributed-systems problem wearing new clothes.

**Clocks.** Coupling a simulation to hardware forces simulation time to follow wall-clock time, making the execution time of a single simulation iteration a critical parameter [23]. A plant model that takes 90 seconds to compute a 60-second step will never close the loop.

**Update cadence.** Zhang et al. put both sides of the cost plainly: too frequent updates burn communication and computation, while delayed updates let the model drift from reality [24]. Heithoff et al. add that consuming everything the sensors produce would stop the software delivering results in meaningful time [3]. Your probe's sampling rate is a design decision, not a device property, and the link carrying the samples is itself engineered [25].

**Trust.** The simulation-reality gap is named as the main challenge across worked case studies [26], and organisations often cannot tell how well a twin matches reality or whether to rely on it for critical decisions [27]. If the twin waters the plant while you are away, that judgement is yours.

## 5. Twins inside the development pipeline

A twin is a long-lived deployed system, and the literature treats its operations accordingly. TwinOps connects twins to DevOps practice [28]; Barbie tests twin prototypes through automated integration tests in a CI pipeline, communication protocols included, with a smart-farming case study and open artefacts for replication [29]; Dalibor et al. map the software engineering literature on twins across domains [30]. Twins also change under new

requirements, which motivates notation for their continuous evolution [31] and automation of their lifecycle management [32]. Kamburjan et al. frame twin and counterpart together as a self-adaptive system: a greenhouse plant that becomes infected needs a different watering regime, so the physical system's stage changes which twin components apply [33].

## 6. Comparison: where to start

| Starting point | Citekey | Core idea | Stated limitation |
|---|---|---|---|
| Architecture pattern catalog | `tekinerdogan_systematic_2021` | A catalog of twin architecture patterns to instantiate | Patterns for structure; behaviour is addressed separately |
| Behaviour pattern catalog | `lehner_pattern_2023` | Augments twin models with behaviour via model-driven engineering | Assumes a modelling-layer/realisation-layer split |
| Entanglement-aware middleware | `bellavista_entanglement-aware_2024` | Middleware that keeps twin and asset coupled, rather than storing twin state | Contrasts itself with commercial platforms it does not replace |
| Serverless/Kubernetes platform | `wermann_ktwin_2024` | Cloud-native tooling, open standards for models | Prototype-stage evaluation |
| Open-source framework survey | `gil_survey_2024` | Compares frameworks through one case study | Single case limits generalisation |
| DTaaS platform | `talasila_realising_2024`, `talasila_composable_2025` | Operator- and tenant-facing platform hosting many twins built from shared parts | Reuse depends on components existing for your assets |
| Plant-and-pump exemplar | `kamburjan_greenhouse_2024` | Plant, sensors and water pumps as the physical system; asset model in a knowledge base | A deliberately simple, low-cost greenhouse |
| Tutorial-grade exemplar | `gomes_digital_2025` | Incubator twin using ordinary differential equations, Kalman filtering and a service-oriented architecture | Single case study |
| Closed-loop exemplar | `goffi_engineering_2025` | Direct control of a physical process as a finite state machine | Domain-specific process constraints |

For this use case, GreenhouseDT is the closest published match: an extensible architecture whose physical system is plants, sensors and water pumps, with the asset model held in the twin's knowledge base [34].

## 7. Gaps in this corpus

Three gaps stand out. First, scale: after reformulating queries across agriculture, greenhouse, sensor-actuator and small-exemplar phrasings, only a handful of sources sit at consumer scale [19], [34]–[36]. Manufacturing and infrastructure dominate, so guidance about one pot is extrapolated from factories. Second, the disagreement in §1 is unresolved in the literature, not merely unsettled here: whether an automated write path to the physical side is required separates Kritzinger's grading [1], [2] from the Digital Twin Consortium's synchronisation-only definition [8], and no retrieved source reconciles them. Third, no retrieved source addresses the running cost — compute, energy, maintenance — of keeping a twin synchronised over years, which for a single plant may exceed the value of the decisions it makes.

## References


[1] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital Twin in manufacturing: A categorical literature review and classification," *Ifac-PapersOnline*, vol. 51, no. 11, pp. 1016–1022, 2018, Accessed: Dec. 11, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896318316021

[2] C. Ellwein *et al.*, "Rethinking Asset Administration Shell Communication Types: A Systematic Mapping Study and Portfolio-Based Classification," *Production Engineering*, vol. 20, Dec. 2025, doi: 10.1007/s11740-025-01378-3.

[3] M. Heithoff, N. Jansen, J. Michael, F. Rademacher, and B. Rumpe, "Model-Based Engineering of Multi-Purpose Digital Twins in Manufacturing," in *Digital Twin: Fundamentals and Applications*, S. Sabri, K. Alexandridis, and N. Lee, Eds., Cham: Springer Nature Switzerland, 2024, pp. 89–126. doi: 10.1007/978-3-031-67778-6_5.

[4] T. Bergs, S. Gierlings, T. Auerbach, A. Klink, D. Schraknepper, and T. Augspurger, "The concept of digital twin and digital shadow in manufacturing," *Procedia CIRP*, vol. 101, pp. 81–84, 2021, Accessed: Jan. 21, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212827121006612

[5] E. VanDerHorn and S. Mahadevan, "Digital Twin: Generalization, characterization and implementation," *Decision Support Systems*, vol. 145, p. 113524, Jun. 2021, doi: 10.1016/j.dss.2021.113524.

[6] C. Semeraro, M. Lezoche, H. Panetto, and M. Dassisti, "Digital twin paradigm: A systematic literature review," *Computers in Industry*, vol. 130, p. 103469, Sep. 2021, doi: 10.1016/j.compind.2021.103469.

[7] R. Minerva, G. M. Lee, and N. Crespi, "Digital twin in the IoT context: A survey on technical features, scenarios, and architectural models," *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1785–1824, 2020, Accessed: Feb. 25, 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9120192/

[8] R. Bhandal, "Conceptualising the Application of Digital Twins in Supply Chain Management: A Path Towards Supply Chain Resilience," in *Digital Twin: Fundamentals and Applications*, S. Sabri, K. Alexandridis, and N. Lee, Eds., Cham: Springer Nature Switzerland, 2024, pp. 173–189. doi: 10.1007/978-3-031-67778-6_8.

[9] E. Ferko, A. Bucaioni, and M. Behnam, "Architecting Digital Twins," *IEEE Access*, vol. 10, pp. 50335–50350, 2022, doi: 10.1109/ACCESS.2022.3172964.

[10] B. Tekinerdogan and C. Verdouw, "Systems Architecture Design Pattern Catalog for Developing Digital Twins," *Sensors*, vol. 20, no. 18, p. 5103, Jan. 2020, doi: 10.3390/s20185103.

[11] D. Lehner, S. Sint, M. Eisenberg, and M. Wimmer, "A pattern catalog for augmenting Digital Twin models with behavior," *at - Automatisierungstechnik*, vol. 71, no. 6, pp. 423–443, Jun. 2023, doi: 10.1515/auto-2022-0144.

[12] P. Bellavista, N. Bicocchi, M. Fogli, C. Giannelli, M. Mamei, and M. Picone, "An Entanglement-Aware Middleware for Digital Twins," *ACM Trans. Internet Things*, Oct. 2024, doi: 10.1145/3699520.

[13] P. Bellavista, N. Bicocchi, M. Fogli, C. Giannelli, M. Mamei, and M. Picone, "Exploiting microservices and serverless for Digital Twins in the cloud-to-edge continuum," *Future Generation Computer Systems*, vol. 157, pp. 275–287, Aug. 2024, doi: 10.1016/j.future.2024.03.052.

[14] A. G. Wermann and J. A. Wickboldt, "KTWIN: A Serverless Kubernetes-based Digital Twin Platform." arXiv, Aug. 2024. doi: 10.48550/arXiv.2408.01635.

[15] A. Barbone, S. Burattini, M. Martinelli, M. Picone, A. Ricci, and A. Virdis, "Digital Twin Continuum: A Key Enabler for Pervasive Cyber-Physical Environments," in *2024 33rd International Conference on Computer Communications and Networks (ICCCN)*, Jul. 2024, pp. 1–9. doi: 10.1109/ICCCN61486.2024.10637565.

[16] C. Alcaraz and J. Lopez, "Digital Twin: A Comprehensive Survey of Security Threats," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 3, pp. 1475–1503, 2022, doi: 10.1109/COMST.2022.3171465.

[17] T. Kulik, Z. Kazemi, and P. G. Larsen, "Security and Privacy-related Issues in a Digital Twin Context," in *The Engineering of Digital Twins*, J. Fitzgerald, C. Gomes, and P. G. Larsen, Eds., Cham: Springer International Publishing, 2024, pp. 313–344. doi: 10.1007/978-3-031-66719-0_13.

[18] P. Talasila, P. H. Mikkelsen, S. Gil, and P. G. Larsen, "Realising Digital Twins," in *The Engineering of Digital Twins*, J. Fitzgerald, C. Gomes, and P. G. Larsen, Eds., Cham: Springer International Publishing, 2024, pp. 225–256. doi: 10.1007/978-3-031-66719-0_11.

[19] P. Talasila *et al.*, "Composable digital twins on Digital Twin as a Service platform," *SIMULATION*, vol. 101, no. 3, pp. 287–311, Mar. 2025, doi: 10.1177/00375497241298653.

[20] S. Gil, P. H. Mikkelsen, C. Gomes, and P. G. Larsen, "Survey on open-source digital twin frameworks–A case study approach," *Software: Practice and Experience*, vol. 54, no. 6, pp. 929–960, Jun. 2024, doi: 10.1002/spe.3305.

[21] P. Zech, C. Nardin, S. Ristov, M. Flora, and R. Breu, "Digital-Twins-as-a-Service in Construction Engineering," in *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*, Aug. 2024, pp. 3004–3010. doi: 10.1109/CASE59546.2024.10711409.

[22] K. Duran *et al.*, "Toward Digital Twin-as-a-Service (DTaaS) Platforms: A Survey on Architecture, Design Requirements, and Performance Metrics," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1845–1878, 2026, doi: 10.1109/COMST.2025.3635582.

[23] M. Frasheri *et al.*, "Addressing time discrepancy between digital and physical twins," *Robotics and Autonomous Systems*, vol. 161, p. 104347, Mar. 2023, doi: 10.1016/j.robot.2022.104347.

[24] N. Zhang, R. Bahsoon, N. Tziritas, and G. Theodoropoulos, "Knowledge Equivalence in Digital Twins of Intelligent Systems," *ACM Transactions on Modeling and Computer Simulation*, vol. 34, no. 1, pp. 1–37, Jan. 2024, doi: 10.1145/3635306.

[25] C. Gomes, D. E. L. Rötter, A. Iosifidis, H. Feng, H. Ejersbo, and M. Frasheri, "Sensing and Communication of Data from the Physical Twin," in *The Engineering of Digital Twins*, J. Fitzgerald, C. Gomes, and P. G. Larsen, Eds., Cham: Springer International Publishing, 2024, pp. 147–171. doi: 10.1007/978-3-031-66719-0_7.

[26] B. J. Oakes *et al.*, "Case Studies in Digital Twins," in *The Engineering of Digital Twins*, J. Fitzgerald, C. Gomes, and P. G. Larsen, Eds., Cham: Springer International Publishing, 2024, pp. 257–310. doi: 10.1007/978-3-031-66719-0_12.

[27] *Foundational Research Gaps and Future Directions for Digital Twins*. Washington, D.C.: National Academies Press, 2024. doi: 10.17226/26894.

[28] J. Hugues, J. Yankel, J. Hudak, and A. Hristozov, "Twinops: Digital twins meets devops," *CARNEGIE-MELLON UNIV PITTSBURGH PA, Tech. Rep.*, 2022, Accessed: Jan. 08, 2025. [Online]. Available: https://apps.dtic.mil/sti/trecms/pdf/AD1168471.pdf

[29] A. Barbie and W. Hasselbring, "Toward Reproducibility of Digital Twin Research: Exemplified with the PiCar-X." arXiv, Aug. 2024. doi: 10.48550/arXiv.2408.13866.

[30] M. Dalibor *et al.*, "A Cross-Domain Systematic Mapping Study on Software Engineering for Digital Twins," *Journal of Systems and Software*, vol. 193, p. 111361, Nov. 2022, doi: 10.1016/j.jss.2022.111361.

[31] J. Mertens, S. Klikovits, F. Bordeleau, J. Denil, and Ø. Haugen, "Continuous Evolution of Digital Twins using the DarTwin Notation." arXiv, Oct. 2024. Accessed: Nov. 11, 2024. [Online]. Available: http://arxiv.org/abs/2410.23389

[32] G. Beaumont, A. Beugnard, S. Martínez, C. Urtado, and S. Vauttier, "Towards Automating the Life Cycle Management of Digital Twins," in *ER2025-44th International Conference on Conceptual Modeling*, 2025. Accessed: Oct. 22, 2025. [Online]. Available: https://hal.science/hal-05225783/

[33] E. Kamburjan, N. Bencomo, S. L. Tapia Tarifa, and E. B. Johnsen, "Declarative Lifecycle Management in Digital Twins," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, Linz Austria: ACM, Sep. 2024, pp. 353–363. doi: 10.1145/3652620.3688248.

[34] E. Kamburjan *et al.*, "GreenhouseDT: An Exemplar for Digital Twins," in *Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, in SEAMS '24. New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 175–181. doi: 10.1145/3643915.3644108.

[35] P.-E. Goffi, R. Tremblay, and B. Oakes, "Engineering a Digital Twin for the Monitoring and Control of Beer Fermentation Sampling." arXiv, Aug. 2025. doi: 10.48550/arXiv.2508.18452.

[36] C. Gomes *et al.*, "Digital Twin Tutorial: The Incubator Case Study," in *Engineering Trustworthy Software Systems: 6th International School, SETSS 2024, Chongqing, China, April 14–21, 2024, Tutorial Lectures*, J. P. Bowen, C. Gomes, and Z. Liu, Eds., Singapore: Springer Nature, 2025, pp. 68–101. doi: 10.1007/978-981-96-4656-2_3.